

HAR Final Report

基于惯性传感器的可穿戴人体活动识别系统 —— 从传感器标定到端侧部署的全链路工程实践

课程论文 | 《模式识别与统计学习》专周实训
孔子瑜（算法与工程）、李宝平（数据采集与固件） | 导师：郑积仕

摘要

人体活动识别（HAR）是可穿戴计算与健康监护的核心技术。本文完成了一套从硬件设计、传感器标定、固件开发、特征工程、模型训练到 ESP32 端侧部署的全链路 HAR 系统。基于 ESP32-S3 + MPU6050 六轴 IMU 硬件平台，以 50Hz 采样加速度与角速度数据，采集 7 类日常活动（静坐、站立、步行、跑步、上楼、下楼、跌倒），共 363 个滑动窗口。经六位置加速度标定（RMSE 11.35→0.42 m/s²）、巴特沃斯滤波与滑窗切分后，提取 84 维手工时域特征。采用逻辑回归（OVR + Softmax）分类，全数据训练精度 100%。模型参数导出为 MicroPython 模块，部署至 ESP32-S3 端侧实时推理（<2ms/次）。系统支持 LED 七色指示与 HTTP Dashboard 远程监控。另外在 849 窗口三被试数据集上进行了时序平滑、1D-CNN 和无监督聚类等进阶探索，多数表决时序平滑 L=7 使 kNN 准确率从 41.7% 提升至 46.1%。

关键词：人体活动识别；模式识别；MEMS 惯性传感器；端侧部署；逻辑回归

1 引言

1.1 研究背景

随着 MEMS 惯性传感器在可穿戴设备中的普及，人体活动识别（Human Activity Recognition, HAR）在健康监护（跌倒检测）、运动量化、智能家居和康复医疗等领域具有广泛应用价值。

大多数 HAR 研究集中于算法层面，使用公开数据集离线训练模型，但忽略了一个关键问题：**模型训好了怎么部署到真实的嵌入式设备上？**从 Python 到 MicroPython、从 numpy 到纯 for 循环、从训练环境到真实传感器——这条部署链路中的每一个工程细节，都可能让一个 99% 的模型在端侧变成随机猜测。

本课题的核心价值在于：从零到一跑通了从硬件 PCB 设计到端侧部署的全链路，并将训练与推理的数学表达严格对齐，最终在 ESP32-S3 上实现实时推理。

1.2 本文贡献

- 1. 完成从 PCB 设计、传感器标定、固件开发、数据采集、特征工程、模型训练到端侧部署的全链路工程闭环
- 2. 系统解决了三类关键工程问题：传感器量程不一致、特征计算训练-推理偏差（ZCR/median/IQR）、嵌入式独立运行稳定性
- 3. 在 849 窗口数据集上完成时序平滑、1D-CNN 和无监督聚类的进阶对比分析
- 4. 实现端侧实时推理系统：MicroPython 固件 + LED 指示 + HTTP/串口 Dashboard

2 系统硬件

2.1 硬件架构

组件	型号/规格
主控	ESP32-S3 N16R8（双核 240MHz, 8MB PSRAM, 16MB Flash）
传感器	GY-521 (MPU6050)：3轴加速度计 + 3轴陀螺仪
通信	I ² C 总线：SDA=IO17, SCL=IO18
指示	NeoPixel RGB LED @ GPIO48
PCB	KiCad 原理图 → Gerber → 嘉立创打样 → 手工焊接

2.2 传感器标定

加速度计六位置标定

将传感器分别以 ±X、±Y、±Z 六个方向静止放置，每位置采集 5 秒，以静止时合加速度应等于重力加速度 $g \approx 9.80665 \text{ m/s}^2$ 为约束，最小二乘求解零偏(bias)和灵敏度(scale)：

$$a_{\text{calibrated}} = (a_{\text{raw}} - \text{bias}) \times \text{scale}$$

结果：RMSE 从 11.35 m/s^2 降至 0.42 m/s^2 （改善 96.3%）。

标定参数写入 `calib/calib_params.json`，离线采集和实时推理固件均加载使用。

2.3 固件架构

固件	路径	功能
离线采集 v2	<code>firmware/offline_logger/main.py</code>	7活动×3次=21 trial 自动序列，LED颜色引导，边采边写 CSV

2026/6/26 17:09

HAR Final Report

固件	路径	功能
实时推理	firmware/realtime_inference/main_wifi.py	50Hz 读取→128 样本窗口→84 维特征→LR推理→LED+HTTP 输出

3 数据采集与预处理

3.1 采集协议

参数	设定值
传感器配置	±8g / ±2000°/s, 50Hz, DLPF 5Hz（寄存器显式配置）
佩戴位置	腰部，绑紧不滑动
被试	S01
活动类别	静坐(0), 站立(1), 步行(2), 跑步(3), 上楼(4), 下楼(5), 跌倒(6)
每类时长	前 6 类 30s, 跌倒 15s
每类次数	3 次
总 trial	21
总窗口	363（128 样本/窗口, 50% 重叠）

3.2 离线采集固件

LED 灯光信号自动引导采集流程：- 颜色慢闪 3 秒（提示下一个动作）→ 暗闪倒计时 8 秒 → 常亮录制 → 白闪完成 - 蓝=静坐, 青=站立, 绿=步行, 黄=跑步, 橙=上楼, 紫=下楼, 红=跌倒 - I²C 错误自动恢复（自建 SoftI2C + 3次重试）

3.3 预处理管道

raw CSV → 标定校准 (ACC_BIAS/ACC_SCALE) → 去重力 (HP 0.3Hz)
→ 低通滤波 (LP 20Hz) → 滑窗 (128样本, 50%重叠) → 窗口标签

4 特征工程

4.1 84 维时域特征

ESP32 MicroPython 无法使用 scipy/numpy，选择纯加减乘除可实现的时域特征。每条轴 14 个统计量 × 6 轴 (ax/ay/az/gx/gy/gz) = 84 维：

特征	说明
mean, std, var, rms	信号基本统计
peak_to_peak, max, min	极值
median($\approx$ mean)	训练时用 mean 替代，避免排序
skew, kurtosis	分布形状
zero_cross_rate	减均值后过零检测
sma	信号幅值面积
iqr($\approx 1.349\sigma$)	训练时用 1.349σ 替代
autocorr_lag1	一阶自相关

4.2 训练-推理数学对齐（核心）

特征	Python 训练 (scipy)	固件推理 (MicroPython)	对齐方式
median	<code>np.median()</code>	<code>mean</code>	训练数据用 mean 覆盖
IQR	<code>scipy.stats.iqr()</code>	<code>1.349 * std</code>	训练数据用 1.349σ 覆盖
ZCR	<code>x-mean(x) -> signbit</code>	<code>x-mean(x) -> sign</code>	固件加 mu 参数

ZCR 是发现并修复的最隐蔽 bug：固件最初直接对原始值检测过零，加速度含重力 ~ 9.8 导致信号从不跨零，6 个 ZCR 特征恒为 0，模型收到的输入完全偏离训练分布。

5 模型与端侧部署

5.1 模型选择：逻辑回归

参数	值
模型	LogisticRegression (OVR + Softmax)
输入	84 维标准化时域特征
参数量	$84 \times 7 = 588$ 权重 + 7 偏置 + 168 标准化参数
训练精度	100% （同人自用，363 窗口全覆盖）

为什么选逻辑回归而不是深度学习？ - ESP32 MicroPython 无法运行 TensorFlow/PyTorch - 588 次乘法 + Softmax < 2ms - 手工特征已提取充足判别信息，线性分类器足够

5.2 模型导出与部署

```
# 训练: sklearn -> 提取 coef_, intercept_, mean_, scale_ -> 写入 model_params.py
# 推理: ESP32 直接 import -> 手写循环
```

```

for i in range(84):
    x[i] = (feat[i] - MEAN[i]) / STD[i]      # 标准化

for c in range(7):
    s = INTERCEPT[c]

    for i in range(84): s += x[i] * COEF[c][i] # 点积

    logits[c] = s

```

→ Softmax → argmax → 输出类别 + 置信度

时序平滑：5 帧多数投票消除单帧抖动。

5.3 实时推理系统

- **LED**：NeoPixel 七色实时指示（蓝=坐, 青=站, 绿=走, 黄=跑, 橙=上楼, 紫=下楼, 红=跌倒）
- **串口 Dashboard**：USB 连接电脑 → python dashboard/app.py → 浏览器实时查看
- **HTTP API**：/api/status 返回 JSON（活动+置信度+传感器波形）

6 工程问题与解决

6.1 传感器量程不一致

问题：离线采集固件配置 $\pm 8g/\pm 2000^\circ/s$ ，实时推理固件使用默认 $\pm 2g/\pm 250^\circ/s$ ，但换算系数仍按 $\pm 8g$ 计算 → 加速度偏小 4 倍，陀螺偏小 8 倍 → 特征完全偏离。

解决：统一两个固件显式写入全部配置寄存器（PWR_MGMT1 + SMPLRT_DIV + CONFIG + ACCEL_CONFIG + GYRO_CONFIG）。

6.2 过零率计算不一致

问题：Python scipy 先减均值再统计过零；固件对原始值统计 → 重力致信号全正 → ZCR 恒为 0 → 6 个特征废掉。

解决：固件 `_zcr(x, mu)` 先减均值再检测过零，与 scipy 行为完全一致。

6.3 充电宝独立运行卡死

问题：USB 连接电脑正常，换充电宝供电后蓝灯→绿灯卡住不动。根因：(1) WiFi 禁用时 `socket.bind('0.0.0.0', 80)` 无网络接口下无限阻塞；(2) 软重启后 MPU6050 I²C 状态机残留。

解决：(1) ESP32_IP='0.0.0.0' 时跳过 `http_init()`；(2) `init_mpu()` 自建新 SoftI2C + 读 WHO_AM_I 验证 + 3 次重试；(3) 加阶段 LED 指示灯精确定位卡点。

7 进阶探索

在 849 窗口三被试数据集（S01+S02+S03）上使用 kNN(k=3,distance)作为基模型进行三项探索：

7.1 时序平滑

方法	Accuracy	Macro F1	sit	walk	upstairs	fall
kNN 无平滑	41.7%	0.399	0%	55.8%	76.8%	28.6%
MV L=5	45.0%	0.439	0%	63.0%	81.2%	33.3%
MV L=7	46.1%	0.446	0%	64.5%	84.1%	33.3%
HMM Viterbi	42.9%	0.411	0%	58.0%	81.2%	28.6%

结论：多数表决 L=7 最佳，Accuracy +4.4%。sit 无法通过时序平滑修复（特征级系统性混淆）。HMM Viterbi 效果不如简单多数投票（转移矩阵从混淆矩阵估计，噪声大）。

7.2 1D-CNN 端到端

三层 Conv1D + BatchNorm + GlobalAvgPool，输入 (128,9) 原始窗口。

结果：Accuracy 16.3%，模型坍缩为全判 sit（其他类全 0%）。849 窗口对 CNN 严重不足。**手工特征+传统分类器在当前数据规模下远优于端到端深度学习。**

7.3 无监督聚类

KMeans / GMM 在原始 292 维和 PCA-50 维空间的聚类评估：

方法	ARI	NMI	AMI
KMeans (292D)	0.061	0.156	0.143
GMM (292D)	0.057	0.174	0.162
KMeans+PCA	0.149	0.304	0.296
GMM+PCA	0.131	0.294	0.285

结论：PCA 降维后聚类显著提升。ARI < 0.15，各类特征空间重叠严重，sit 窗口散布在多个 cluster 中——与监督学习 sit recall=0% 相互印证。**sit 是全局性难题。**

8 总结与展望

8.1 工作总结

- 1. 硬件层：ESP32-S3 Shield PCB 设计→打样→焊接→I²C 验证→固件开发

2. **数据层**：离线采集固件 v2（21 trial 自动序列 + LED 引导）→ 363 窗口
3. **特征层**：84 维手工时域特征 + 训练-推理数学表达精确对齐（ZCR/median/IQR）
4. **模型层**：逻辑回归（OVR+Softmax）→ 全数据精度 100%→ 导出 MicroPython 参数
5. **部署层**：ESP32 端侧实时推理（<2ms）→ LED 七色 + HTTP/串口 Dashboard
6. **探索层**：时序平滑（L=7, +4.4%）+ CNN（不适合小样本）+ 聚类（sit 全局难题）

8.2 核心工程洞察

- **嵌入式中每个数学操作都要对**：ZCR 少一个减均值，6 个特征全废
- **传感器配置要显式**：依赖默认值会在不同供电场景下出莫名其妙的 bug
- **训练和推理必须完全一致**：不是思路一致，是逐行代码一致
- **端侧部署是系统工程**：不是训好模型扔过去就行，I²C 时序、内存限制、软硬复位全要处理

8.3 未来方向

1. 多被试数据（5+ 人）提升泛化性，补充 sit 类别样本量
2. 时序平滑 + 姿态角特征 + 域自适应缓解跨被试偏移
3. 数据扩充后重新评估 1D-CNN
4. RF/GBDT 集成方法在小样本下的对比

8.4 个人收获

孔子瑜（算法与工程）：完成了从”调包跑模型”到”真懂算法部署全链路”的转变。最有价值的是工程排错——传感器量程不对、ZCR 计算不一致、充电宝卡死——每个问题的定位和解决都加深了对嵌入式 AI 系统的理解。

李宝平（数据采集与固件）：掌握了 I²C 总线调试、传感器标定操作、ESP32 MicroPython 固件开发全流程。固件在 USB 供电正常但充电宝卡死的问题排查，深刻理解了嵌入式设备脱离调试环境后的独立运行要求。

致谢

感谢郑积仕老师在项目全程中的悉心指导，特别感谢老师在传感器标定方法、LOSO 评估必要性和端侧部署方面的关键建议。

参考文献

1. 张学工.《模式识别》（第四版）. 清华大学出版社, 2021.
2. 李航.《统计学习方法》（第二版）. 清华大学出版社, 2019.

3. Bulling, A., Blanke, U., Schiele, B. “A Tutorial on Human Activity Recognition Using Body-worn Inertial Sensors.” *ACM Computing Surveys*, 46(3): 1-33, 2014.

4. Anguita, D., Ghio, A., Oneto, L., Parra, X., Reyes-Ortiz, J.L. “A Public Domain Dataset for Human Activity Recognition Using Smartphones.” *ESANN*, 2013.

5. Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” *JMLR*, 12: 2825-2830, 2011.

6. Ordóñez, F.J., Roggen, D. “Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition.” *Sensors*, 16(1): 115, 2016.

附录 A. 核心代码清单

文件	行数	职责
src/preprocess.py	503	信号预处理管道
src/features.py	448	294维特征提取
src/feature_selection.py	619	特征选择与降维
src/classifiers_baseline.py	403	基线分类器
src/models_advanced.py	347	高级模型与调参
src/evaluation.py	509	模型评估
src/d9_advanced_exploration.py	814	进阶探索
src/d11_export_model.py	235	模型导出
firmware/offline_logger/main.py	280	离线采集固件 v2
firmware/realtime_inference/main_wifi.py	580	实时推理固件

附录 B. 图表索引

图表	文件
传感器标定对比	reports/figures/accel_calib_comparison.png
预处理总览	reports/figures/preprocess_overview.png
t-SNE 特征可视化	reports/d5_tsne.png
特征重要性	reports/d5_rf_importance.png
特征选择方法对比	reports/d5_method_comparison.png
模型对比	reports/d7_model_comparison.png
混淆矩阵	reports/d8_confusion_matrix.png
ROC 曲线	reports/d8_roc_curves.png
模型排名	reports/d8_model_ranking.png
时序平滑对比	reports/d9_temporal_smoothing_recall.png
HMM 转移矩阵	reports/d9_hmm_transition_matrix.png

图表

文件

CNN 对比	reports/d9_cnn_comparison.png
聚类 t-SNE	reports/d9_clustering_tsne.png
聚类指标	reports/d9_clustering_metrics.png

版本: v2.0 | 2026-06-26 | 全链路完成, 端侧部署验证